



AIUB SOLUTION

[Follow us]



Like Our Facebook Page: <https://www.facebook.com/AIUBSolution/>

Join Our Facebook Group: <https://www.facebook.com/groups/solutionaiub>

Watch Our Video Tutorial on Youtube:

<https://www.youtube.com/c/AIUBSOLUTION-AIUB/>



[AIUB SOLUTION GROUP]



[AIUB SOLUTION YOUTUBE]



[AIUB SOLUTION PAGE]

Regular expression

A sequence of symbols and character expressing a string or pattern to be searched for within longer piece of text, mainly used for pattern matching

Recursive $\Rightarrow$ বাবা বাবা repeat করা

3 Rules:

- (i) Basic
- (ii) Induction
- (iii) Precedence

Rules of Regular expression: (4 types of induction)

$$(P) (S) \Rightarrow L(P) \cup L(S)$$

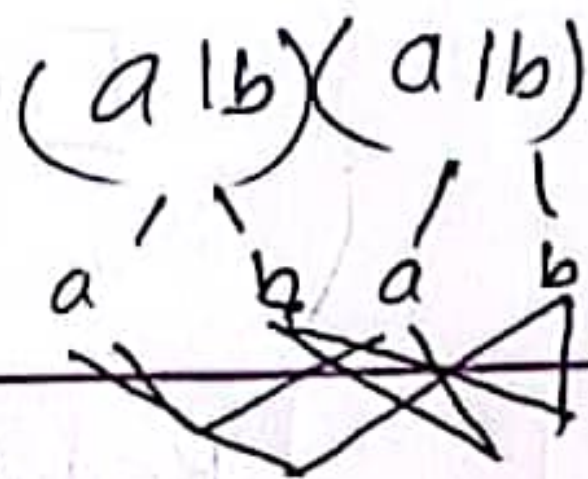
$$(P) (S) \Rightarrow L(P) \cdot L(S)$$

$(P)^*$ $\Rightarrow$ occurrence, 0 to many frequency

$L(P)$ $\Rightarrow$ Regular exp

we can add ^{pair of} parenthesis around expression,

এই language এর লেখন change আছে না।



Let, $E \in \{a, b\}$

1. r.e $a|b$ denotes language $\{a, b\}$

2. $(a|b)(a|b)$ denotes $\{aa, ab, ba, bb\}$

3. $\Rightarrow a|b|a|b|b|bb \Rightarrow$ अक्षर को 0R दिए जावा त

Represent कर सकते हैं

$(a^*) = 0$ to many occurrence of a .

multiple a ,

0 a .

it accepts the empty string.

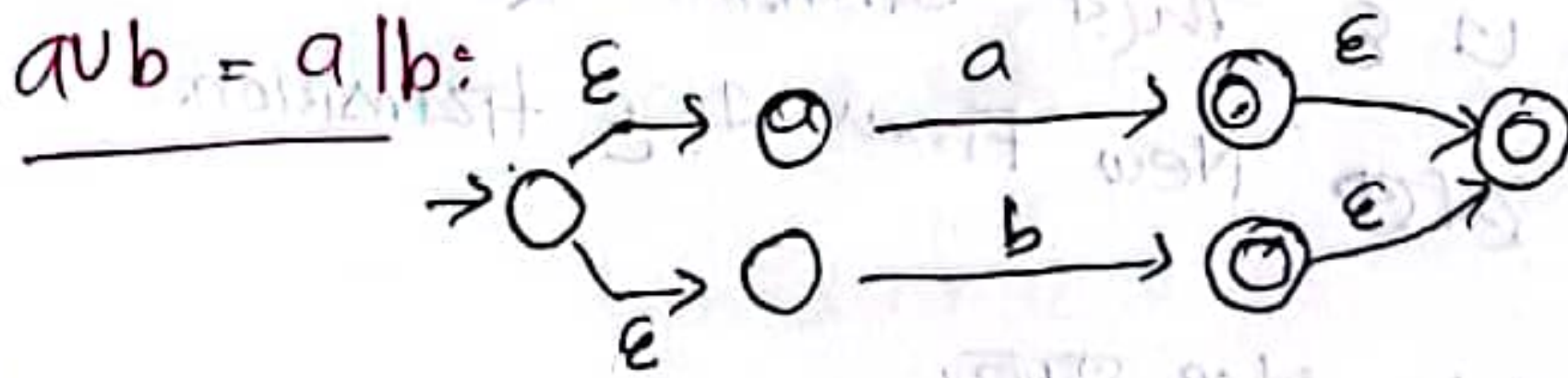
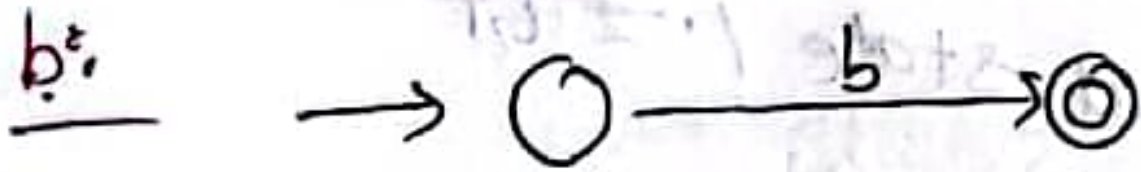
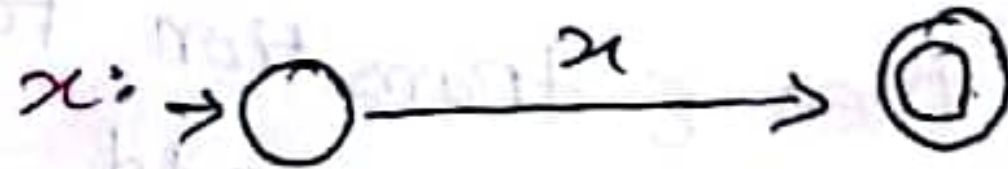
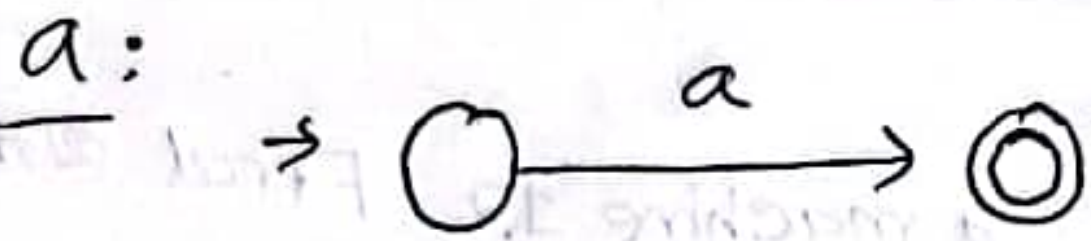
ab^*

$\{b=0, a\}$

4. $(a|b)^* = \{ \epsilon, a, b, aa, ab, ba, bb, aaa, \dots \}$

5. $(a^*b^*)^*$

6. $a|a^*b = \{a, b, abb, aaab, \dots\}$ starts one or more but ending is b .



Last common step

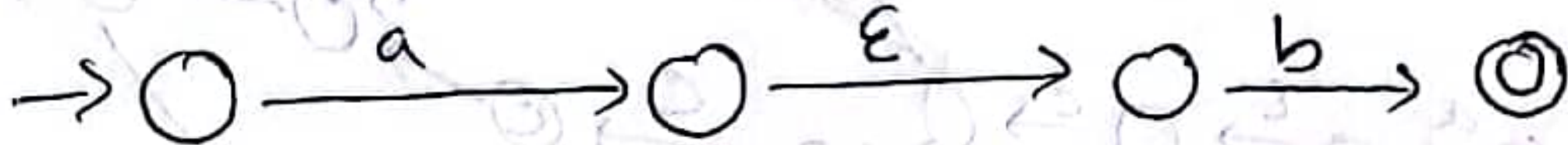
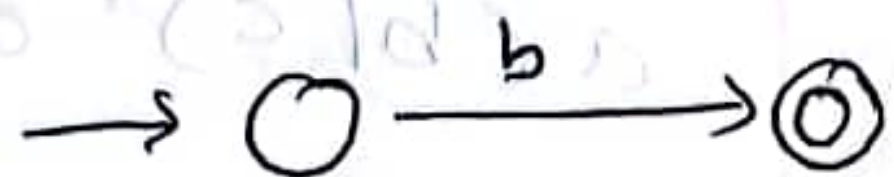
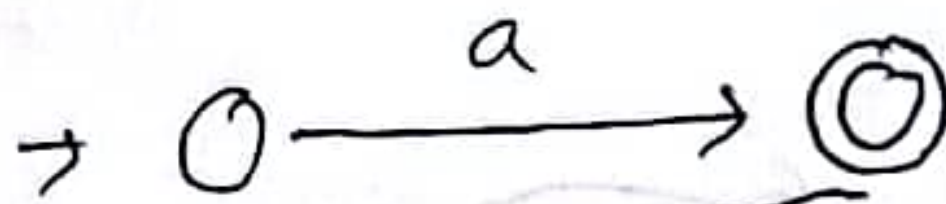
দিয়ে

যদি এক কিছু দেয়া হয়
প্রাকার না পড়লে ε use
করবে।

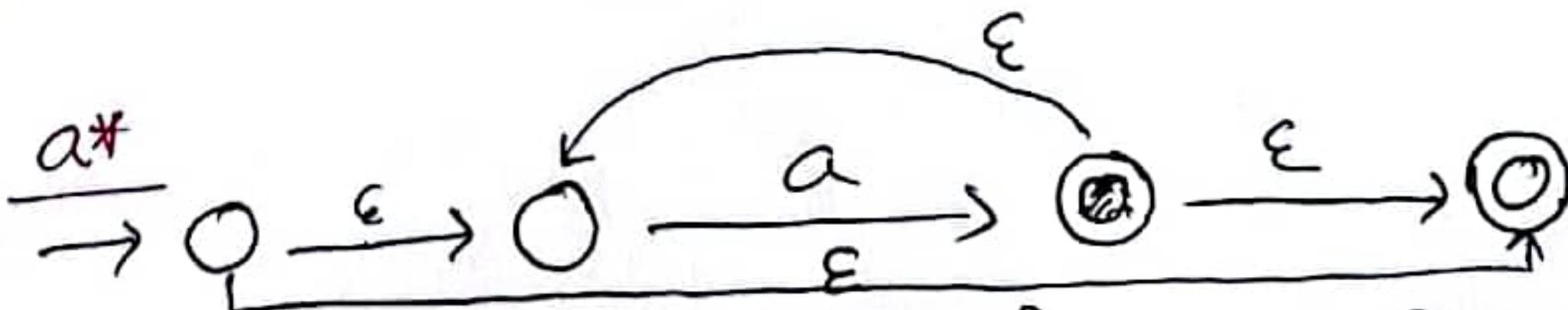
$$a|b = b|a$$

but $ab \neq ba$ | কারণ আগে b দেয়া
পরে a দেয়া

ab:



* Final state ko Normal state
* ε দিয়ে transition.

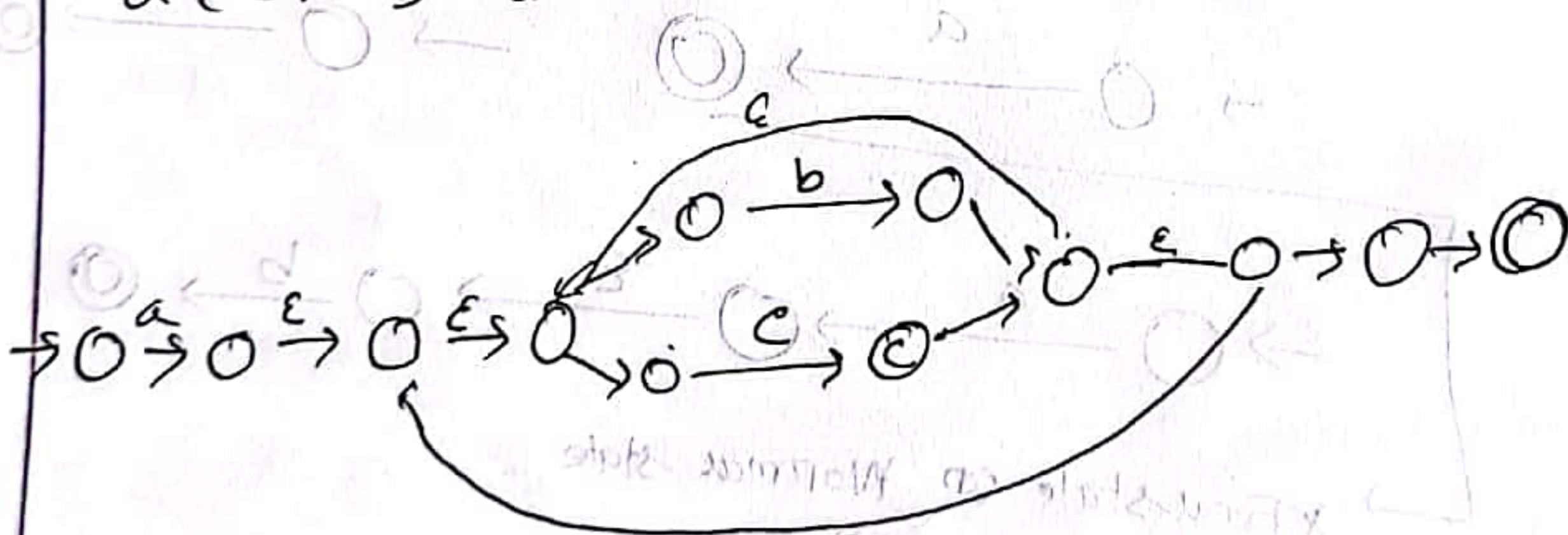


যদি 1 বার, 2 বার, n বার কিছু কোন কিছু দেয়া না করে,
তাহলে ε দিয়ে কার্য।

- $\Rightarrow$ first a machine design
 $\Rightarrow$ then ϵ transition দিয়া a machine 17 Final state start
 $\Rightarrow$ New final add
 $\Rightarrow$ Transit ϵ , make final state 1. ঠিক
 $\Rightarrow$ Start step 17 ϵ দিয়ে আবার নতুন start.
 $\Rightarrow$ New start ঠিক New final 17 ϵ transition.
 # total 4 ta step আছে।

Thompson Law: Regular exp $\rightarrow$ NFA Convert an

$a(b|c)^*d$



Program file x

Code Blocks

MinGW → bin → link → copy →

This is Right click → Property → type environment variable

Advance → Environment variable.

Path → New/ Edit → setup.

download gnu32win32

flex, Bison.

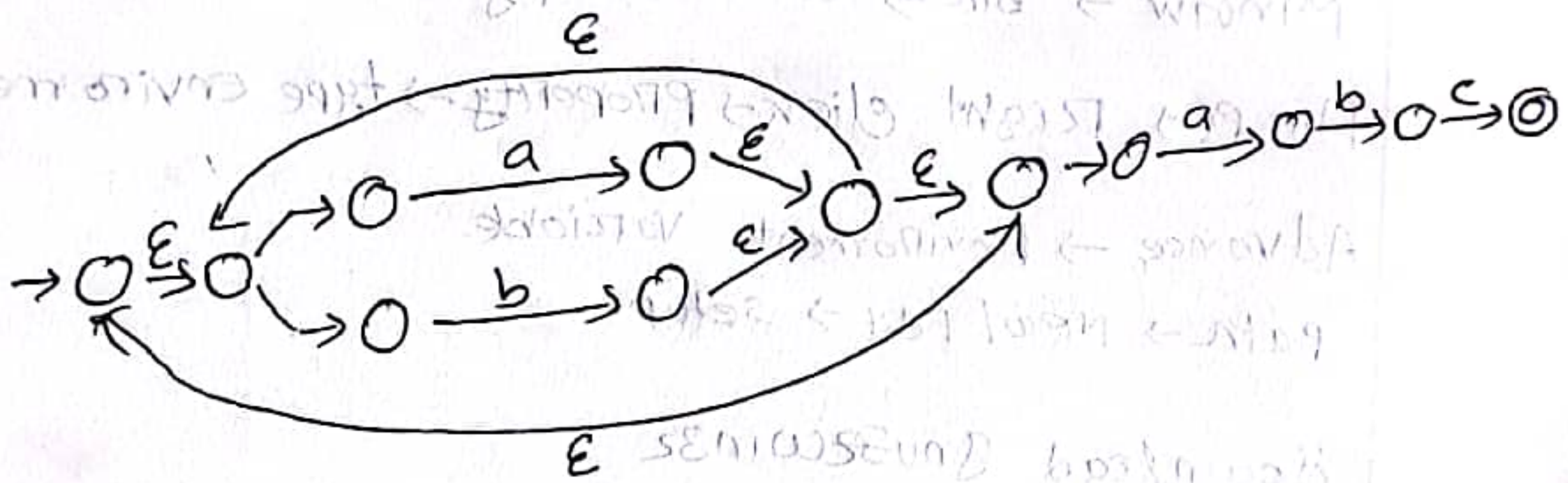
flex hello.c

~~gcc lex.yy.c~~ gcc lex.yy.c

a.exe

class exercise

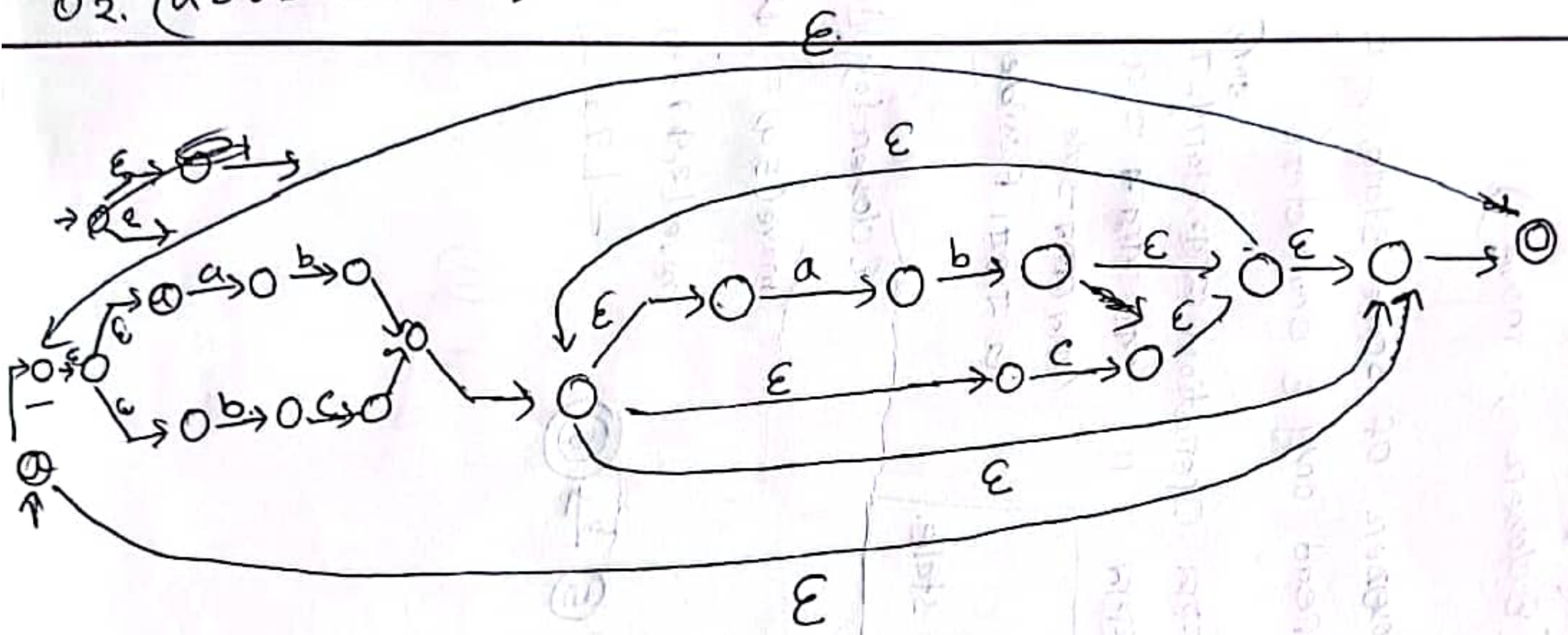
1. $(a \cup b)^* abc$



2. $(ab \cup bc(ab \cup c))^*$

$\rightarrow 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{c} 0$
 $\rightarrow 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{c} 0$
 $\rightarrow 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{c} 0$

02. $(abvbdabvc)^*)^*$



New start mining



NFA to DFA Conversion

Subset Construction method

(mainly 2)

we use 3 method, (E closer, move)

1. E closer (s) $\Rightarrow$ E closer of some state. $\Rightarrow$ we read only E character

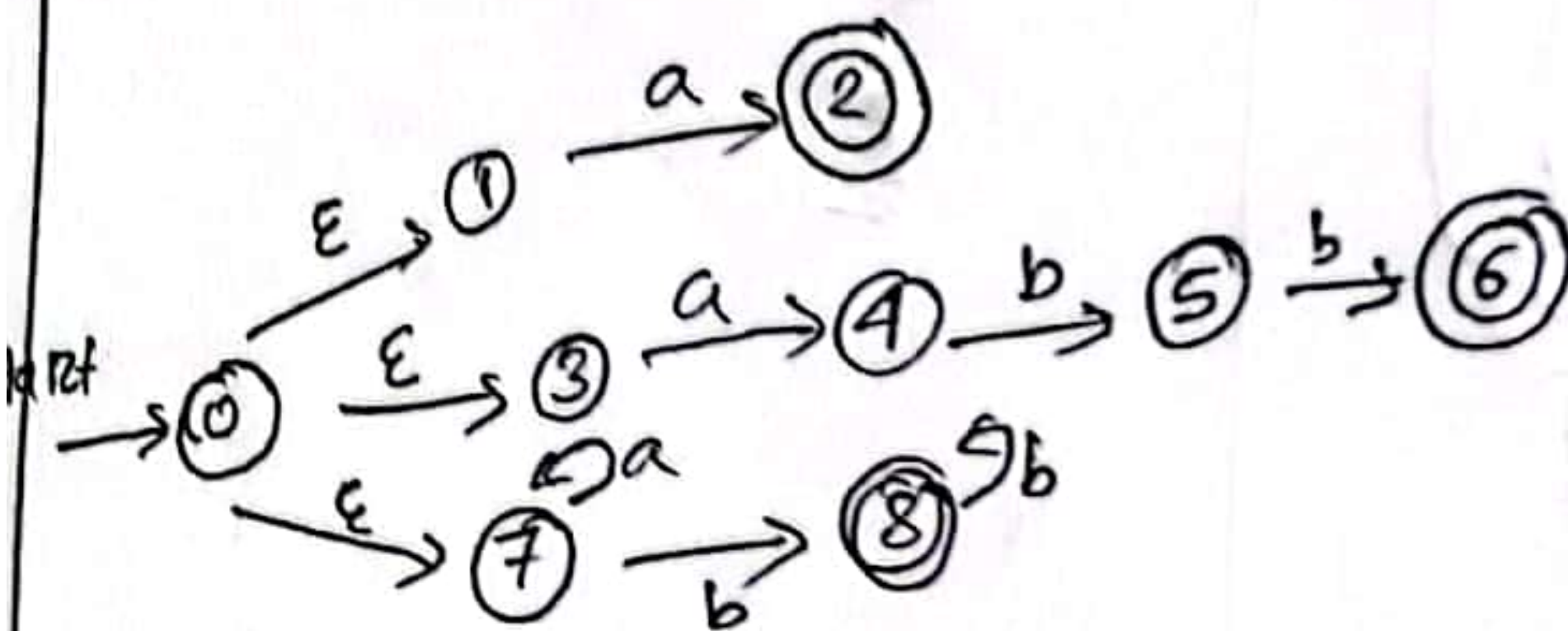
2. move (T, a) $\Rightarrow$ কোন Operation থেকে state T
লগন $\parallel$ বের করুন = a.
কোন কোন state
এ যাওয়া possible

include the current state
Exclude the current state.

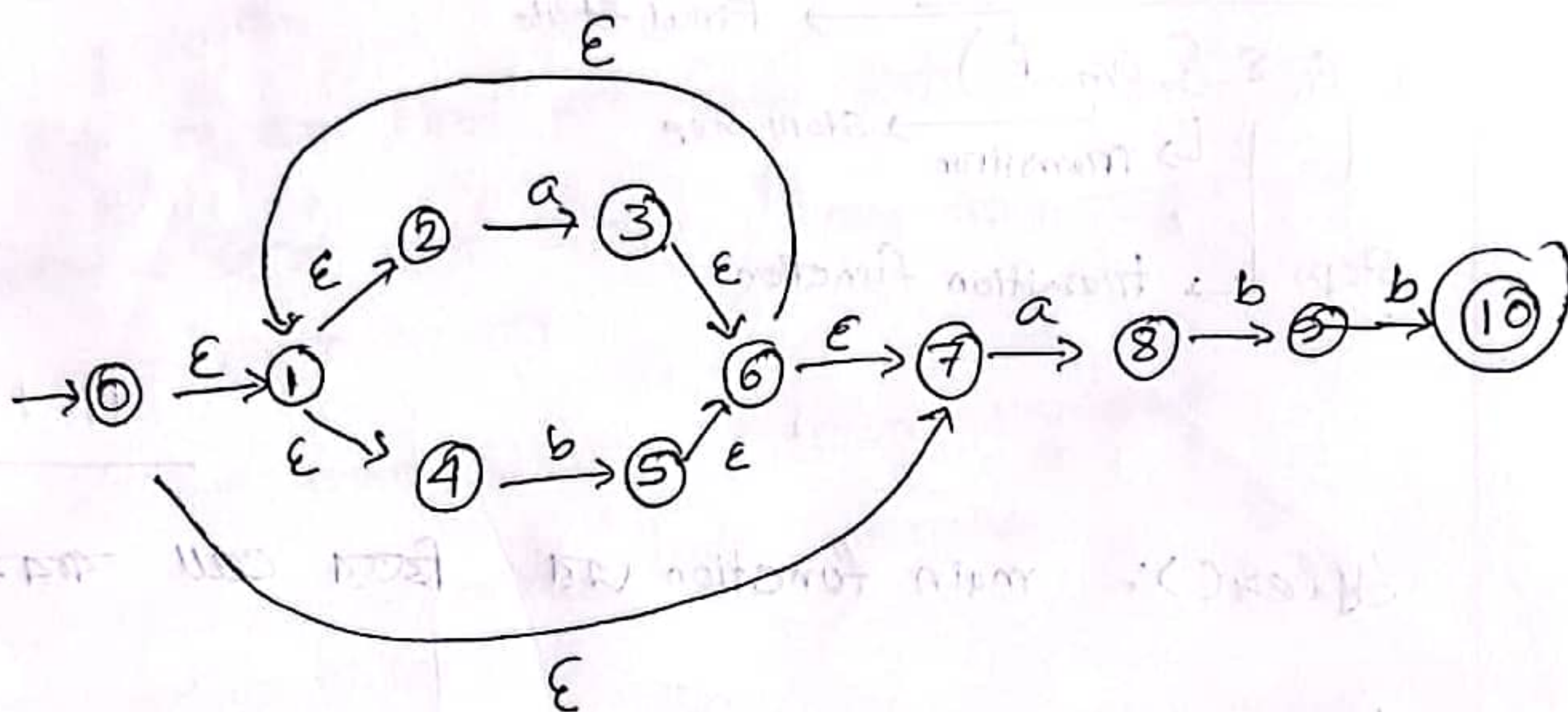
E closer {0, 1, 3, 7}

move(5, a) = {4}

move({2, 4, 7}, a)
= {7}



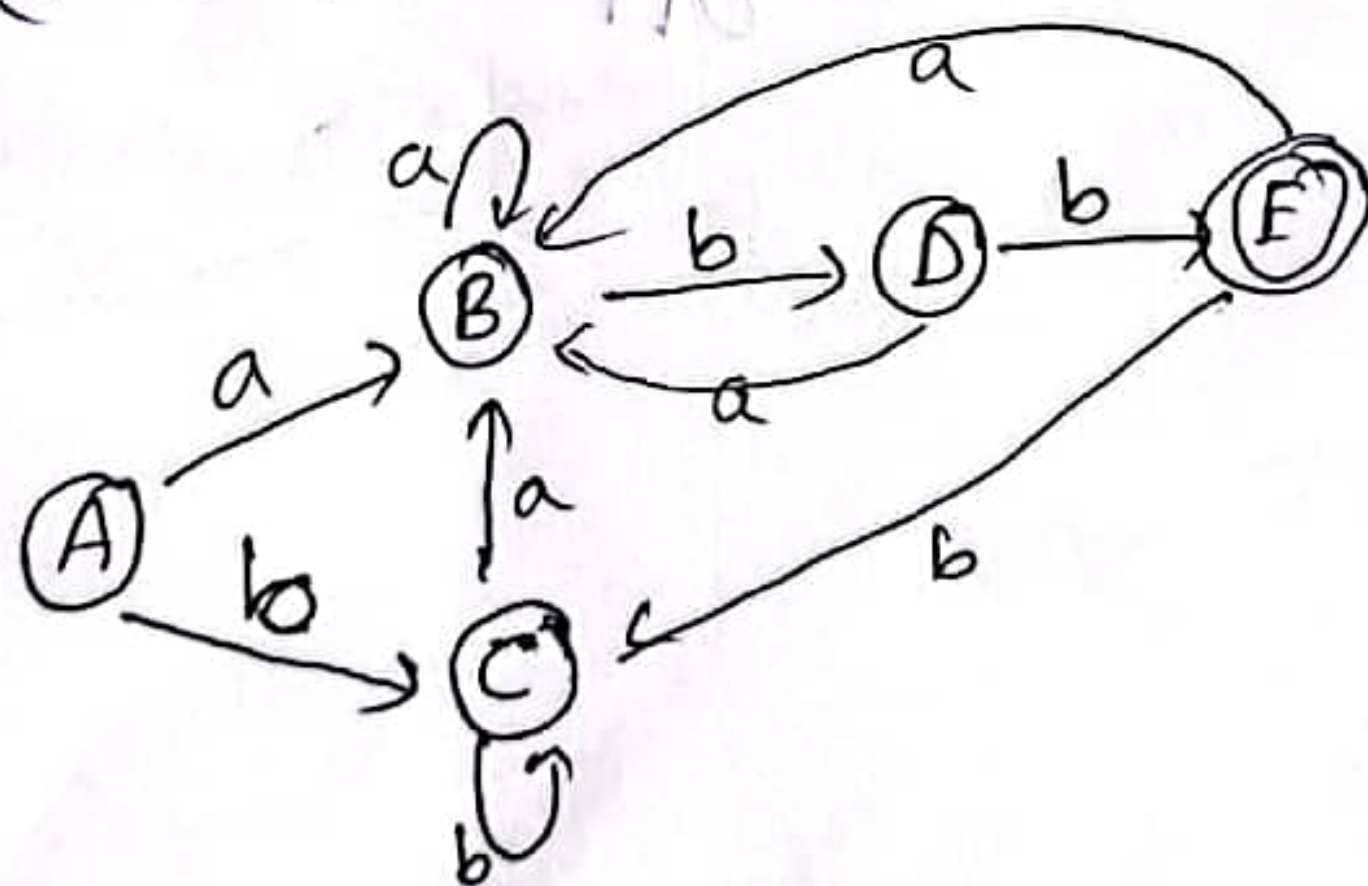
NFA ত same start end না বলেও ক সম্ভাব্য নাই।



সম্পন্ন দেয়া আছে,

NFA এর নং ১ (৬) অক্ষর যাবে যে state দেয়াছি তা start step.

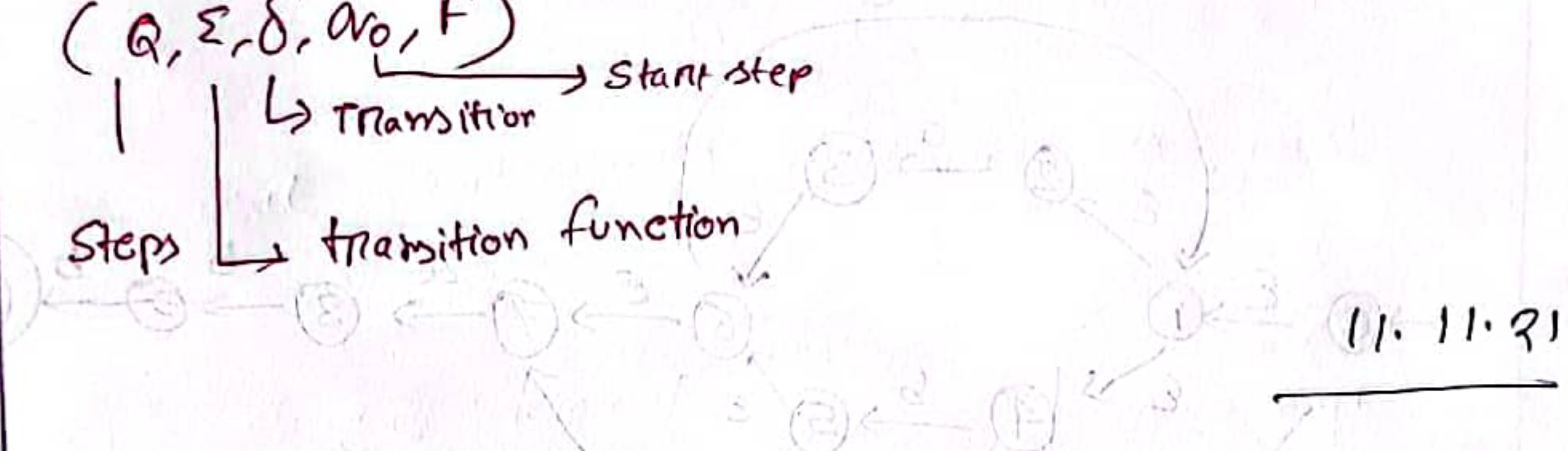
NFA এর 10 যে যে step নং আছে, সেটিকে Final step



DFA has 5 Tuple.

$(Q, \Sigma, \delta, q_0, F)$
 → Final State
 → Start step
 → Transition

Steps → transition function



bylexC); main function এর ডিটা call করতে

by

111000

001100

011



First and follow

very
easy

bit Complicate

←

CFG এর জন্য First and follow বোঝা যায়,

First = প্রথম, শুরু পূর্ব } যেখানে CFG দে

Follow = শেষ বা পরে

CFG এর main content = term = variable, terminal, rules.

set of non-terminal aka variable
production rules.

First and follow for all nonterminal / variable

এর জন্য

variable detection: product rule এর জন্য যা শাকলে, তাই

The basic problem in parsing is choosing

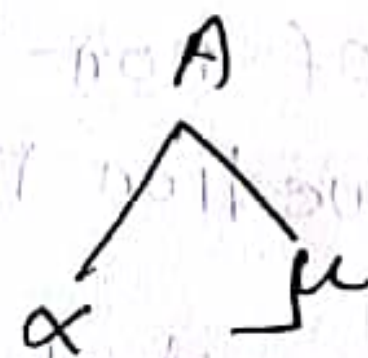
First Set (Case -1)

$\Rightarrow$ For a production, if the first thing is terminal, that would consider as First

$\Rightarrow$ Left thing is terminal, then that terminal will be First.

$\Rightarrow$ Don't worry abt rest

$$A \rightarrow \alpha \mu$$



$$\text{First}(A) = \{ \alpha \}$$

Case: 02

$\Rightarrow$ Production, first thing (ϵ) First will be (ϵ)

$$A \rightarrow \epsilon = \{ \epsilon \}$$

$$A \rightarrow \epsilon abc$$

$$\rightarrow \{ a \}$$

$$\text{First}(A) = \{ \epsilon \}$$

Case: 03

If they non-terminal / variable, we should continue for & until we found a terminal.

$A \rightarrow (BCDE)$ Then language B is 1st set of the language

Example: of

 $\in \mathbb{Z}$

↑ First

$$\begin{aligned} E &\rightarrow TE \\ E &\rightarrow \underline{TE'} \mid \epsilon \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$
$$E \Rightarrow S$$

4/10/2012
SAT

$\rightarrow T \rightarrow (C, T')$ $\rightarrow (Id, T')$

LF is Corresponding

$$\text{First}(F) = \{ \epsilon, id \}$$

Varziabla,

Tg? Rules.

$$T \Rightarrow FT'$$

Non fermi

First char निम्न

एक फीस का बिना चालान

Lid, Trj

$\Rightarrow T = ($

$\text{id } T' = \text{id}$

T

$E' = \{ T, \epsilon \}$

ચરણ તાશ

T ના કોઈ Rules ને નથી,

E ના Rules ને નથી,

$\text{First}(T) = \{ (, \text{id} \}$

$\text{First}(E') = \{ T, \epsilon \}$

Follow

Follow એટલે કે જો સમાન variable નો છે

$E \rightarrow T E'$

$E' \rightarrow T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

i) work with variable, right side only

ii) start from starting variable

iii) starting variable નો First નો \$ add કરવો

v) Follow જોઈને મેળવો Find out first out first of all non terminal

$$\text{Follow}(E) = \{ \$,) \}$$

$\downarrow$ (E)

$\downarrow$ এর ফলাফল

সব rule
check করে
F (সমস্ত
(E)).

$$\text{Follow}(E') = \text{Follow}(E) = \{ \$,) \}$$

E এর পরে কে
আছে,

যদি scenario এমন হয় যার কোন variable
আছে যদি আর কোন variable
না থাকে, তাহলে Last যে যা দিয়ে
defined হয়েছে সেটাকে দিয়ে derivation
করা যাবে।

left sideward
 $TE' + TE'$

$$\text{Follow}(T) = \{ +, \$, () \}$$

$$TE' = T +$$

$$T \rightarrow TE$$

T এর পর E/
E এর first set

$\rightarrow$ E এর first T
 $\rightarrow$ E' এর follow

Flex.

yytext $\Rightarrow$ user define ya data define

Rules section:

1. Pattern,
2. action against it.

[A-z] { printf ("%s Capital letter \n" ; yytext)

17.11.21

identifiers,

operators

keyword

number of semicolon

line no,

Parsing: CFA, যদি input কে generate করতে পারে, তবে বলা হয় parsing.

Parsing table diff category হতে পারে।

যেমন: Top down, Bottom up etc

→ Leaf to Root

3 structure এর

Parse tree.

with the root.

Top হতে Root

পাশ থেকে leaf

Root → Leaf level.

Time Complexity: $O(n^3)$

2 type Parser

(i) Top down: Root to leaves

(ii) Bottom up: leaves to root

Parsing table collect information from first and follow.

LL 1 Grammar ⇒ $LL1 =$ → input at a time
 ↳ Left to right parsing ↳ Left most derivative

$S \rightarrow SS' \mid \epsilon$
 $S' \rightarrow a, b, \epsilon$

$S \rightarrow \epsilon$
 $S' \rightarrow b, a$

Pre requisit:

(i) Make suitable for top down parser. & Use Elimination of Left recursion and performing

left factoring

Rules for

Removing
left recursion

left side

निराक वा

वा का खतरा।

2. Find first and follow
3. Parsing table generate.

Example:

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

→ No left recursion

→ So first rules done.

Rules: 02:

First and follow set see at a week. first

last

3. ଯଦି variable ଠିକ୍ Row.

Column ହେଉଛି set of terminals

ଫଲୋ
ଏବଂ \$

	+	*	(	)	id	\$
E	-	-	$E \rightarrow TE'$	-	$E \rightarrow TE'$	-
E'	$E' \rightarrow +TE'$	-	-	$E' \rightarrow \epsilon$	-	$E' \rightarrow \epsilon$
T	-	-	$T \rightarrow FT'$	-	$T \rightarrow FT'$	-
T'	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$				$T' \rightarrow \epsilon$
F				$F \rightarrow (E)$	$F \rightarrow id$	

ସାଧାରଣ ଟର୍ମିନାଲ୍

Setup:

E ଓ First and follow:

E ଓ First ଥିବା

E ଆଗରେ TE' ଥାଏ,

ଯେଉଁଠି ୨ ଆସୁ ନାହିଁ Rule ନିର୍ଦ୍ଦେଶିତ, ସାମାନ୍ୟତା

ଫଲୋ intersection.

$E' \rightarrow$ First $\{+, E\}$

+ ଓ ଆଗରେ $+TE'$

E ଓ ଉପରେ ନାହିଁ, ଯଦି follow ଥାଏ ବାକି

check for empty cell.

যদি per cell 1 Multiple Rules আছে, তাহলে এটি অসঙ্গত
হবে ambiguous.

এক cell 1 Multiple Rules হওয়া non-ambiguous

Slide: 10

$S \rightarrow A$
 $A \rightarrow aB \mid Ad$
 $B \rightarrow bB \mid cf$
 $C \rightarrow \epsilon$

Step: 1

$A \rightarrow Ad \mid ab$

$A \rightarrow Ad$

LR

$A \rightarrow aBA'$

$A' \rightarrow dA' \mid \epsilon$

$S \rightarrow A$

$B \rightarrow bB \mid cf$

$C \rightarrow \epsilon$

→ First in First out

17:11
22

Stack Movement Predictive Parser:

id + id

Trace

Stack	Input	Rule
\$, E	(id) + id, \$	
\$ E' (E)	id + id \$	$E \rightarrow TE'$
\$ E' T' (F)	(id) + id \$	$F' \rightarrow FT'$
\$ E' T' (id)	(id) + id \$	$F \rightarrow id$
\$ E' (T)	(+) id \$	
\$ E'	+ id \$	$T' \rightarrow E$
\$ E' T +	+ id \$	$E' \rightarrow +TE'$
\$ E'	id \$	
\$		
\$		
\$		
\$		
\$		
\$	\$	$E' \rightarrow \epsilon$

OR Mathe id,
Mathe id term
E ko push krna
ni chahiye

→ Pop

— This input is generated via this grammar via parsing table

for E, id intersection krna hoga, then rules ko chnana

Rules ko TE' krna, Stack ko push krna, E' ko push krna
Stack ko Top id + id No input

1. Top id ko intersection

2. jo same krni hoga, First ko only Pop No check

* in case ନା ହେବା ନିମ୍ନ * ସାବଧାନ, ତଥ୍ୟ?

$E1, \text{inat}$ এর ক্ষেত্রে $\Rightarrow$ Intersection বোঝায় বর্তমান।
 $\text{Intersection, Blank.} \Rightarrow$ Rules Blank.

Intersection, print.	বলতম এই input, এই গ্রামার
তখন মঞ্জি মঞ্জি	
দিয়ে generate করা	যাবে না।

* জাবান ধরে নিই,
২ টা Rules থাকলে, কোন Rules add করব?

when to end?

When the input is Blank then
stack is empty
end of the program.

1. Make the grammar suitable

False

2. Follow set of γ

(b) $\{h, fg\}$

3. parsing is the second phase

4. In a top down parser, the start

(3) non terminal

5. Follow set of (E)

(3) $\{f, h\}$

6. ϵ can be the element of follow

True

7. $\alpha \beta_1 | \alpha \beta_2$

(3) $A \rightarrow \alpha A'; A' \rightarrow \beta_1 | \beta_2$

8. Follow set of c

(4) $\{h, fg\}$

9. Which phenomenon will happen

(1) Left recursion

Consider $baababab$

10. $hhah$

1. Predictive Parser

$\Rightarrow$ leftmost derivation

Transitional function of DFA

$\Rightarrow QX\Sigma \rightarrow Q$

Follow of C

$\Rightarrow \phi$

Empty strings can be inputted NFA

$\Rightarrow$ True

$CA|Ab|a$ is not suitable because

$\Rightarrow$ left recursive

$E \rightarrow E+|E*E|a$

Suitable context free grammar is

$\Rightarrow E \rightarrow aA; \quad A \rightarrow +EA | *EA | \epsilon$

Corresponding DFA

$\Rightarrow (a|b)^*a$

First of B

$\Rightarrow \{g, \epsilon\}$

Follow of B

$\Rightarrow \{ \$, a, h, g \}$

Parsing table --- grammar

$\Rightarrow$ False

~~$A = \{ d \}$~~

$B = \{ g, \epsilon \}$

$C = \{ h, \epsilon \}$

$A = \{$

$\Rightarrow \{ d, a \}$

$\{ g, \epsilon \}$

~~$\{ g, h, \epsilon \}$~~